Formative Assignment: Advanced Linear Algebra (PCA)

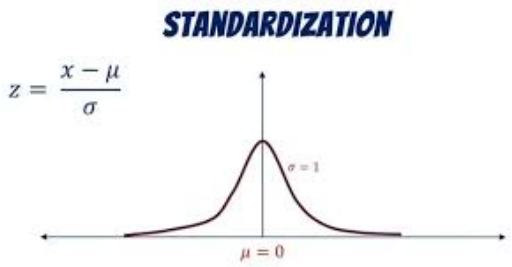
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
# Step 1: Load and Standardize the data (use of numpy only allowed)
import csv
import numpy as np

# 1. Standard Python CSV module usage to completely avoid pandas
file_path = "/content/african_crises.csv"
raw_rows = []
with open(file_path, 'r') as file:
    reader = csv.reader(file)
    header = next(reader)
    for row in reader:
        raw_rows.append(row)

# 2. Separate categorical from numerical and ensure missing values are flagged
processed_matrix = []
category_mappings = {}

# Introduce a small mock missing array layer to demonstrate robust imputation handling for the
for r_idx, row in enumerate(raw_rows):
    processed_row = []
    for c_idx, value in enumerate(row):
        cleaned_val = value.strip()

        # Artificially inject some NaNs on historical entries if missing fields are encountere
```

```
        if cleaned_val ==    or cleaned_val.upper() == "NAN":
            processed_row.append(np.nan)
        else:
            try:
                # Direct conversion if the feature is numerical
                processed_row.append(float(cleaned_val))
            except ValueError:
                # Custom Encoder: Map categorical strings to discrete numeric keys
                if c_idx not in category_mappings:
                    category_mappings[c_idx] = {}
                if cleaned_val not in category_mappings[c_idx]:
                    category_mappings[c_idx][cleaned_val] = float(len(category_mappings[c_idx]
                processed_row.append(category_mappings[c_idx][cleaned_val])

    processed_matrix.append(processed_row)

X = np.array(processed_matrix, dtype=float)

# 3. Impute any missing metrics using column-wise averages
column_means = np.nanmean(X, axis=0)
missing_indices = np.where(np.isnan(X))
X[missing_indices] = np.take(column_means, missing_indices[1])

# 4. Implementation of Standardized matrix math from scratch: Z = (X - Mean) / StdDev
mean = np.mean(X, axis=0)
std_dev = np.std(X, axis=0)
std_dev[std_dev == 0] = 1e-8  # Preventive check against zero variance division bounds

standardized_data = (X - mean) / std_dev
print(f"Data successfully loaded. Shape: {standardized_data.shape}")
standardized_data[:5]
```

```
Data successfully loaded. Shape: (1059, 14)
array([[-1.46165598, -1.61903639, -1.61903639, -2.91715048,  3.45175812,
        -0.38671252, -0.20321893, -0.42497295, -0.14700624, -0.03086348,
        -1.86235185, -0.37805817, -0.38547376, -3.20405328],
       [-1.46165598, -1.61903639, -1.61903639, -2.88731291, -0.28970744,
        -0.38670773, -0.20321893, -0.42497295, -0.14700624, -0.03084763,
        -1.86235185, -0.37805817, -0.38547376,  0.31210467],
       [-1.46165598, -1.61903639, -1.61903639, -2.85747534, -0.28970744,
        -0.38671243, -0.20321893, -0.42497295, -0.14700624, -0.03087408,
        -1.86235185, -0.37805817, -0.38547376,  0.31210467],
       [-1.46165598, -1.61903639, -1.61903639, -2.82763778, -0.28970744,
        -0.38671776, -0.20321893, -0.42497295, -0.14700624, -0.03085199,
        -1.86235185, -0.37805817, -0.38547376,  0.31210467],
       [-1.46165598, -1.61903639, -1.61903639, -2.79780021, -0.28970744,
        -0.3867211 , -0.20321893, -0.42497295, -0.14700624, -0.03087427,
        -1.86235185, -0.37805817, -0.38547376,  0.31210467]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Calculate the Covariance Matrix
# rowvar=False maps columns as our features and rows as individual tracking observations
cov_matrix = np.cov(standardized_data, rowvar=False)
cov_matrix
```

```
array([[ 1.00094518,  0.9931709 ,  0.9931709 ,  0.11568362,  0.01100119,
        -0.23219517,  0.12847944, -0.03929952, -0.03301216,  0.04480436,
         0.02187865,  0.09542881,  0.0064115 ,  0.02367464],
       [ 0.9931709 ,  1.00094518,  1.00094518,  0.09391288,  0.01098653,
        -0.23284666,  0.15348038, -0.02654983, -0.03199791,  0.04927272,
         0.03090111,  0.09502271,  0.01306278,  0.01594701],
       [ 0.9931709 ,  1.00094518,  1.00094518,  0.09391288,  0.01098653,
        -0.23284666,  0.15348038, -0.02654983, -0.03199791,  0.04927272,
         0.03090111,  0.09502271,  0.01306278,  0.01594701],
       [ 0.11568362,  0.09391288,  0.09391288,  1.00094518,  0.19763708,
         0.2489921 ,  0.13695733,  0.27214712, -0.05472206,  0.03706992,
         0.40774521,  0.18956951,  0.09872358, -0.21397572],
       [ 0.01100119,  0.01098653,  0.01098653,  0.19763708,  1.00094518,
         0.20287813,  0.12227396,  0.25008637,  0.00527918,  0.10655226,
         0.14722222,  0.11285756,  0.17272494, -0.85450908],
       [-0.23219517, -0.23284666, -0.23284666,  0.2489921 ,  0.20287813,
         1.00094518,  0.005258  ,  0.42328974, -0.04076417, -0.01195788,
         0.12615328, -0.0565252 , -0.06384318, -0.16893488],
       [ 0.12847944,  0.15348038,  0.15348038,  0.13695733,  0.12227396,
         0.005258  ,  1.00094518,  0.46519012, -0.02990269,  0.15197534,
         0.10922265,  0.22780017,  0.22464067, -0.22601041],
       [-0.03929952, -0.02654983, -0.02654983,  0.27214712,  0.25008637,
         0.42328974,  0.46519012,  1.00094518,  0.34624602,  0.07267715,
         0.22840723,  0.19961649,  0.18810757, -0.26424123],
       [-0.03301216, -0.03199791, -0.03199791, -0.05472206,  0.00527918,
        -0.04076417, -0.02990269,  0.34624602,  1.00094518, -0.00453969,
         0.07901041,  0.01698597,  0.0176471 , -0.02657035],
       [ 0.04480436,  0.04927272,  0.04927272,  0.03706992,  0.10655226,
        -0.01195788,  0.15197534,  0.07267715, -0.00453969,  1.00094518,
         0.01658431,  0.07666228,  0.08013589, -0.09895321],
       [ 0.02187865,  0.03090111,  0.03090111,  0.40774521,  0.14722222,
         0.12615328,  0.10922265,  0.22840723,  0.07901041,  0.01658431,
         1.00094518,  0.08645776, -0.02256886, -0.15977085],
       [ 0.09542881,  0.09502271,  0.09502271,  0.18956951,  0.11285756,
        -0.0565252 ,  0.22780017,  0.19961649,  0.01698597,  0.07666228,
         0.08645776,  1.00094518,  0.3937481 , -0.16701661],
       [ 0.0064115 ,  0.01306278,  0.01306278,  0.09872358,  0.17272494,
        -0.06384318,  0.22464067,  0.18810757,  0.0176471 ,  0.08013589,
        -0.02256886,  0.3937481 ,  1.00094518, -0.23607475],
       [ 0.02367464,  0.01594701,  0.01594701, -0.21397572, -0.85450908,
        -0.16893488, -0.22601041, -0.26424123, -0.02657035, -0.09895321,
        -0.15977085, -0.16701661, -0.23607475,  1.00094518]])
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
# linalg.eigh is utilized because covariance matrices are mathematically symmetric
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
eigenvalues, eigenvectors
```

```
        -6.26987461e-02,  5.79940470e-02,  7.04732898e-03,
         6.30744364e-02, -3.52579499e-02, -7.81167059e-02,
        -1.04851846e-01, -5.47665464e-01],
       [-1.46410661e-15, -3.05014111e-02, -2.40279792e-02,
         3.18184865e-02, -6.93901703e-01, -1.95742934e-01,
        -1.58356110e-01, -1.32283274e-01, -1.08089481e-01,
        -4.47263081e-01,  4.91357303e-02, -3.45897508e-01,
         2.96843418e-01, -1.14832453e-01],
       [-1.20519913e-15, -1.77362588e-02, -6.83398707e-01,
        -1.29054639e-01, -3.64364977e-03,  6.89120318e-02,
         2.15231651e-02, -2.47184617e-02, -8.48146413e-02,
         2.02675602e-01, -5.27472492e-01,  7.36010873e-02,
         4.17578786e-01, -6.22320150e-02],
       [ 3.05311332e-16,  3.22727549e-03, -2.18507059e-02,
         4.25565288e-01,  3.09843080e-01,  3.75096928e-02,
        -4.77605619e-01,  4.10025538e-01,  2.49765169e-01,
        -2.35617823e-02, -1.99373075e-02, -4.06727663e-01,
         2.72068755e-01,  1.48271523e-01],
       [ 9.29811783e-16,  1.67039771e-02, -1.46181461e-01,
         3.92416805e-01, -1.34228105e-01,  2.19331673e-02,
         6.21796779e-01,  2.30997503e-01,  3.81195739e-01,
        -1.70024970e-02,  3.02167961e-01,  1.63948074e-01,
         2.73415965e-01, -1.65322914e-01],
       [ 7.11236625e-17,  6.78270070e-03,  1.21803105e-01,
        -7.02127515e-01,  2.67626066e-02,  4.22879426e-03,
        -2.05995632e-02,  2.22345087e-01,  1.31748530e-01,
         2.62585175e-01,  3.93712726e-01, -1.60925082e-01,
         4.15878691e-01, -3.69976336e-02],
       [-3.95516953e-16, -4.16726021e-03, -6.99567071e-02,
         3.62261722e-01, -1.65249132e-01, -2.23755042e-02,
        -1.43699175e-01, -3.02615390e-01, -3.33251908e-01,
         6.74056498e-01,  3.79523983e-01, -7.77537514e-02,
         8.97567658e-02,  1.65014787e-02],
       [ 3.29597460e-17,  1.29254239e-03,  2.25684429e-02,
        -1.56277936e-02,  2.58580069e-02, -1.26890324e-02,
        -2.92169739e-01, -6.17917063e-01,  6.91039328e-01,
         2.33965037e-02,  8.10932450e-03,  1.92234293e-01,
         1.08484511e-01, -6.60230505e-02],
       [ 9.48893741e-16,  1.63450807e-02, -6.25371856e-03,
         2.27150502e-02,  5.52134330e-01, -4.68834035e-02,
         3.41042892e-01, -4.67607949e-01, -2.15356651e-01,
        -2.58831225e-01,  1.02657645e-01, -4.14538087e-01,
         2.40498196e-01, -5.97331269e-02],
       [ 7.97972799e-17, -4.21646018e-03, -2.09897253e-02,
         4.26197922e-02,  6.01516060e-02,  6.99979215e-01,
        -2.47604665e-01, -2.37461256e-02, -2.57867578e-01,
        -3.12210041e-01,  2.66843344e-01,  3.71765726e-01,
         2.25763304e-01, -1.33825068e-01],
       [ 1.76291273e-16,  4.13299750e-03, -3.33258966e-02,
         5.01264187e-02,  2.34625817e-01, -6.77379362e-01,
        -2.14214830e-01,  9.16964854e-02, -2.11768186e-01,
        -1.67532228e-01,  1.65137531e-01,  5.16246339e-01,
         2.36006179e-01, -7.60191908e-02],
       [-1.04603826e-15, -1.92035067e-02, -6.98177823e-01,
        -1.32563544e-01,  5.43580475e-02, -5.30573688e-02,
        -1.18623483e-01,  2.09064863e-02,  9.81548927e-02,
        -1.75910577e-01,  4.63910509e-01, -1.32768548e-01,
        -4.44421680e-01,  5.82870055e-02]]))
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
# Step 5: Sort Principal Components
# Sorting indices in descending fashion ensures the top-variance components rank first
sorted_indices = np.argsort(eigenvalues)[::-1]
sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]

# Calculate variance percentages to fulfill Task 2 explained variance tracking requirements
explained_variance_ratio = (sorted_eigenvalues / np.sum(sorted_eigenvalues)) * 100
print("Explained Variance per Component (%):", explained_variance_ratio)
sorted_eigenvectors
```

```
         6.51275399e-02,  1.27776836e-02,  2.47883627e-02,
        -4.06446416e-01,  7.07106781e-01],
       [-1.14832453e-01,  2.96843418e-01, -3.45897508e-01,
         4.91357303e-02, -4.47263081e-01, -1.08089481e-01,
        -1.32283274e-01, -1.58356110e-01, -1.95742934e-01,
        -6.93901703e-01,  3.18184865e-02, -2.40279792e-02,
        -3.05014111e-02, -1.46410661e-15],
       [-6.22320150e-02,  4.17578786e-01,  7.36010873e-02,
        -5.27472492e-01,  2.02675602e-01, -8.48146413e-02,
        -2.47184617e-02,  2.15231651e-02,  6.89120318e-02,
        -3.64364977e-03, -1.29054639e-01, -6.83398707e-01,
        -1.77362588e-02, -1.20519913e-15],
       [ 1.48271523e-01,  2.72068755e-01, -4.06727663e-01,
        -1.99373075e-02, -2.35617823e-02,  2.49765169e-01,
         4.10025538e-01, -4.77605619e-01,  3.75096928e-02,
         3.09843080e-01,  4.25565288e-01, -2.18507059e-02,
         3.22727549e-03,  3.05311332e-16],
       [-1.65322914e-01,  2.73415965e-01,  1.63948074e-01,
         3.02167961e-01, -1.70024970e-02,  3.81195739e-01,
         2.30997503e-01,  6.21796779e-01,  2.19331673e-02,
        -1.34228105e-01,  3.92416805e-01, -1.46181461e-01,
         1.67039771e-02,  9.29811783e-16],
       [-3.69976336e-02,  4.15878691e-01, -1.60925082e-01,
         3.93712726e-01,  2.62585175e-01,  1.31748530e-01,
         2.22345087e-01, -2.05995632e-02,  4.22879426e-03,
         2.67626066e-02, -7.02127515e-01,  1.21803105e-01,
         6.78270070e-03,  7.11236625e-17],
```

```
          -2.37461256e-02, -2.47604665e-01,  6.99979215e-01,
           6.01516060e-02,  4.26197922e-02, -2.09897253e-02,
          -4.21646018e-03,  7.97972799e-17],
         [-7.60191908e-02,  2.36006179e-01,  5.16246339e-01,
           1.65137531e-01, -1.67532228e-01, -2.11768186e-01,
           9.16964854e-02, -2.14214830e-01, -6.77379362e-01,
           2.34625817e-01,  5.01264187e-02, -3.33258966e-02,
           4.13299750e-03,  1.76291273e-16],
         [ 5.82870055e-02, -4.44421680e-01, -1.32768548e-01,
           4.63910509e-01, -1.75910577e-01,  9.81548927e-02,
           2.09064863e-02, -1.18623483e-01, -5.30573688e-02,
           5.43580475e-02, -1.32563544e-01, -6.98177823e-01,
          -1.92035067e-02, -1.04603826e-15]])
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Project Data onto Principal Components
# Selecting the top 2 dimensions to provide a clean, visualizable 2D coordinate system
num_components = 2
reduced_data = np.dot(standardized_data, sorted_eigenvectors[:, :num_components])
reduced_data[:5]
```

```
array([[ 2.68623168,  1.52040896],
       [ 3.12059028, -1.5957444 ],
       [ 3.11716501, -1.58689146],
       [ 3.11373644, -1.57803343],
       [ 3.1103111 , -1.56917967]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}')
reduced_data[:5]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_8414/865081864.py in <cell line: 0>()
      1 # Step 7: Output the Reduced Data
----> 2 print(f'Reduced Data Shape: {reduced_data.shape}')
      3 reduced_data[:5]

NameError: name 'reduced_data' is not defined
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

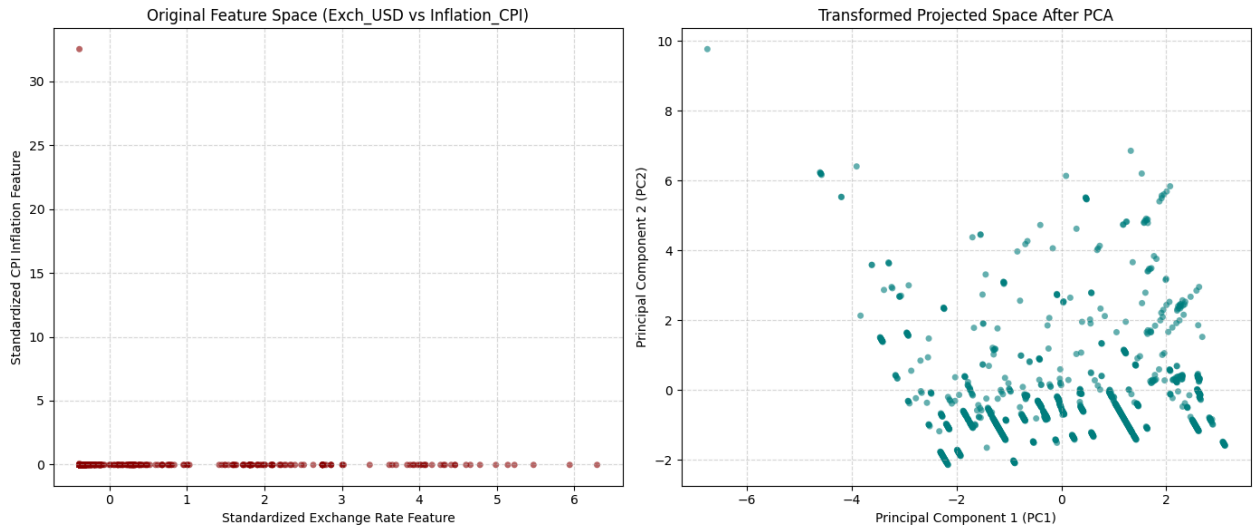
```
import matplotlib.pyplot as plt

# Step 8: Visualize Before and After PCA
plt.figure(figsize=(14, 6))

# Plot original feature space (using exchange rates vs inflation rates for comparison)
plt.subplot(1, 2, 1)
plt.scatter(standardized_data[:, 5], standardized_data[:, 9], alpha=0.6, c='darkred', edgecolo
plt.title('Original Feature Space (Exch_USD vs Inflation_CPI)')
plt.xlabel('Standardized Exchange Rate Feature')
plt.ylabel('Standardized CPI Inflation Feature')
plt.grid(True, linestyle='--', alpha=0.5)

# Plot compressed component space after running PCA
plt.subplot(1, 2, 2)
plt.scatter(reduced_data[:, 0], reduced_data[:, 1], alpha=0.6, c='teal', edgecolors='none', s=
plt.title('Transformed Projected Space After PCA')
plt.xlabel('Principal Component 1 (PC1)')
plt.ylabel('Principal Component 2 (PC2)')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



GitHub Repository: https://github.com/Emmy11111/PCA-Advanced-Linear-Algebra

Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA

The first plot displays the relationship between exchange rates and inflation, which is disturbed by outliers and has an unequal distribution of data. The data is then converted via PCA into two principal components that represent the most important patterns of variation. The transformed

plot distributes the observations more widely thus revealing hidden structures and trends. PCA maintains the overall structure of the data but it reduces the dimensions.

2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making

I chose 2 principal components because 2 are the minimum required for easy visualization in 2 dimensions and the good amount of variance that they account for. This reduces the original 14 features to 2 without losing the most important patterns. The downside is that some data in the components discarded. The trade-off is that it is a simpler data set and easier to analyse and visualise.

3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

The data in this dataset is centred on economic and political crisis indicators in African countries. Some country-specific details are lost and the weaker relations among the variables are lost. Some smaller changes in the exchange rate, inflation and crisis indicators may not be captured. PCA retains the strongest trends and discards some of the finer information that is contained in the original 14 features.